CryptAlyst

Platform Reference — data, bus, models, mathematics and operations

Paper trading only · 27 model books · one RabbitMQ event bus · zero REST polling for market data

Item	Value
Platform	CryptAlyst (scalp5)
Market data	Binance spot, 161 websocket streams, 40 symbols
Transport	RabbitMQ 4.3.6 over STOMP, 19 topics, 19 queues
Execution	Maker simulation, 0.02% per side
Books	27 engines, $5,000 each — $135,000 combined
Dashboard	http://127.0.0.1:8799
Broker UI	http://127.0.0.1:15672

This document describes what the platform does, how each model decides, and the mathematics behind every calculation. It is written to be checkable: each formula appears with the code path that implements it, and each claim is followed by the measurement that supports it.

Every number in this document was measured on the running platform. Where a measurement is weak (small sample, censored data, single venue) it is labelled as weak rather than presented as fact.

Contents

1 Executive summary and evidence

2 Architecture

3 Market data layer

4 The event bus

5 Intelligence: signals, flow, TA, forecast

6 Risk governor

7 Execution and maker fills

8 Books, PnL and accounting

9 Models — cross-sectional and signal ensembles

10 Models — strategies, market-making and execution

11 Measurement: decay, walk-forward, backtest

12 Operations

13 Defect ledger

14 Appendix: mathematics and glossary

1 Executive summary and evidence

CryptAlyst is a paper-trading platform for Binance spot. It collects market data over websockets, publishes everything onto a single RabbitMQ bus, and runs 27 independent model books against the same data. Each book starts at $5,000 and keeps its own equity, positions and trade history.

The design principle that drove the final architecture is simple: collect once, publish once, consume everywhere. No model calls the exchange. No model calls another model's HTTP endpoint for data that already exists on the bus. This was not true for most of the platform's life, and the cost of it being untrue is recorded in section 13.

1.1 What is measured, not asserted

Claim	Measurement	Sample
Market data is streams only	161 websocket streams, 0 REST polls per cycle, 0 limit violations	live
One bus carries everything	19 topics, 19 durable queues, 0 unconsumed, 0 backlog	live
Models read intelligence from the bus	intel cache: 148 symbols, ta 98, forecast 42, depth 248 per cycle	live
Gross edge before costs is near zero	+0.005% per trade	174 closed trades
Cost per trade dominates	-0.1825% per trade (fee + spread + slippage)	174 closed trades
Backtest acceptance	MAR 16.15 pass, drawdown 8.05% pass, expectancy 0.119% FAIL	149 trades, 20 symbols
Winners exit faster than losers	median hold 24s vs 157s	70 fills

1.2 The central finding

Across the platform's history, closed trades averaged a gross result of +0.005% and a net result of -0.177%. The difference is friction: 0.04% maker fee per side, plus the spread crossed on exit, plus slippage. The models were not wrong about direction often enough to pay for the cost of being in the market. That is the single most important fact about this platform, and every subsequent design decision — maker-only execution, a cost floor, staleness gates, the backtest harness — exists because of it.

The remaining sections describe the machinery in full, because a platform that cannot explain its own numbers cannot be improved. Where a model looks profitable, the reason is usually that its losses are not being realised (section 8.3), not that its edge is real.

2 Architecture

The platform is a set of single-purpose processes that share nothing except the bus and a handful of files under state/. Each process can be restarted independently, and the bus preserves the latest message per topic so a restarting consumer catches up immediately.

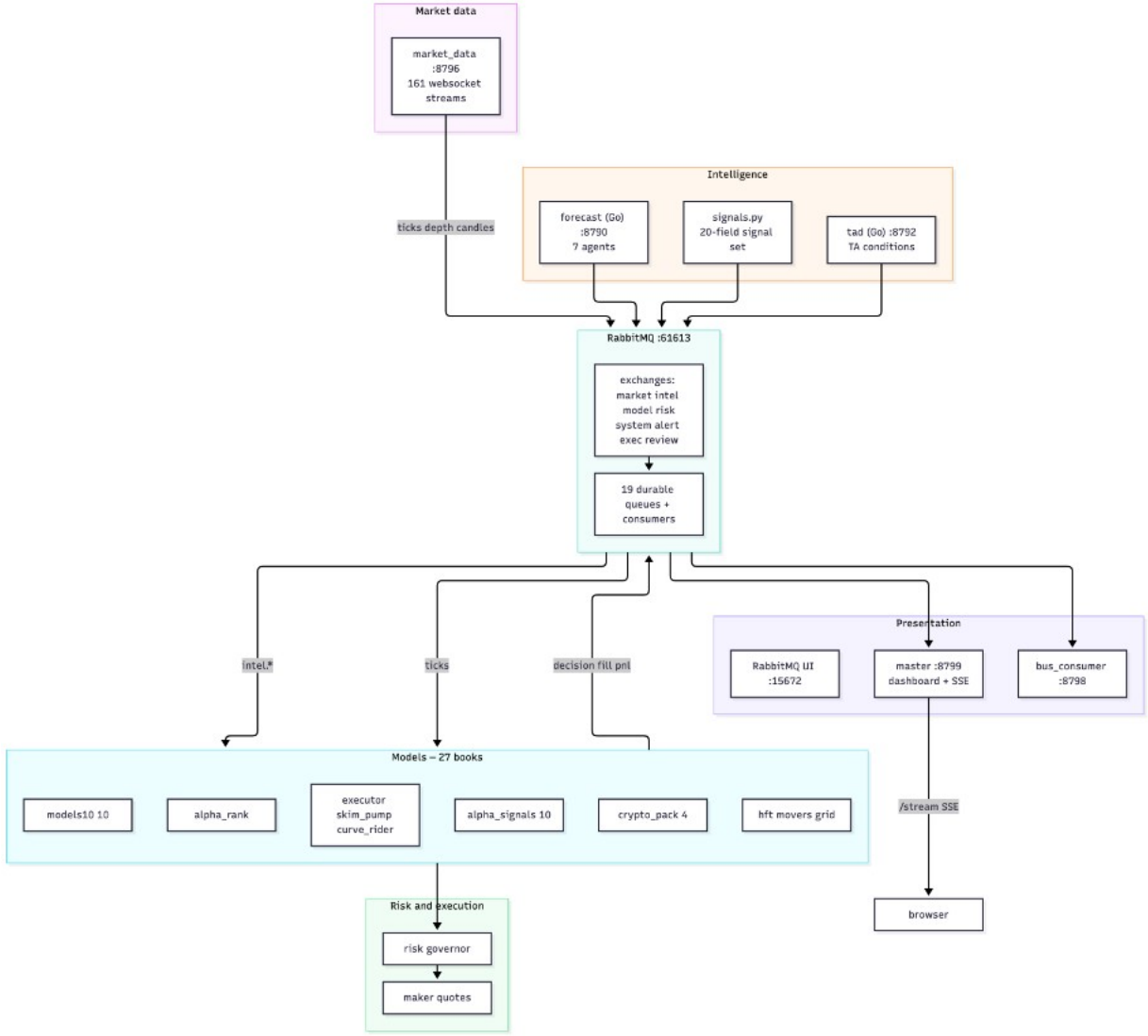


Figure 1 — Services and transport. Every arrow into the models passes through RabbitMQ.

2.1 Process inventory

Process	Port	Language	Responsibility
market_data	8796	Python	161 streams: bookTicker, depth5, kline 1m/15m, miniTicker
signals	—	Python	20-field cross-sectional signal set, 2 topics
tad	8792	Go	TA conditions per symbol, published every cycle

forecast	8790	Go	Seven-agent blend, published on a loop
signal_bus	8793	Python	Pump/dump alert intake, republished to the bus
health_pub	—	Python	system.health every 5s
bus_consumer	8798	Python	One consumer per topic queue, tallies
risk	—	Python library	The governor every model calls before trading
master	8799	Python	Dashboard, SSE hub, 38 API endpoints
supervisor	—	Python	Starts, restarts and prunes books
27 model books	—	Python	Each with its own $5,000 equity

2.2 Request path for one decision

The path below is what actually happens between a price tick and a position. Every arrow is a bus message or a function call inside one process; none of them is an exchange request.

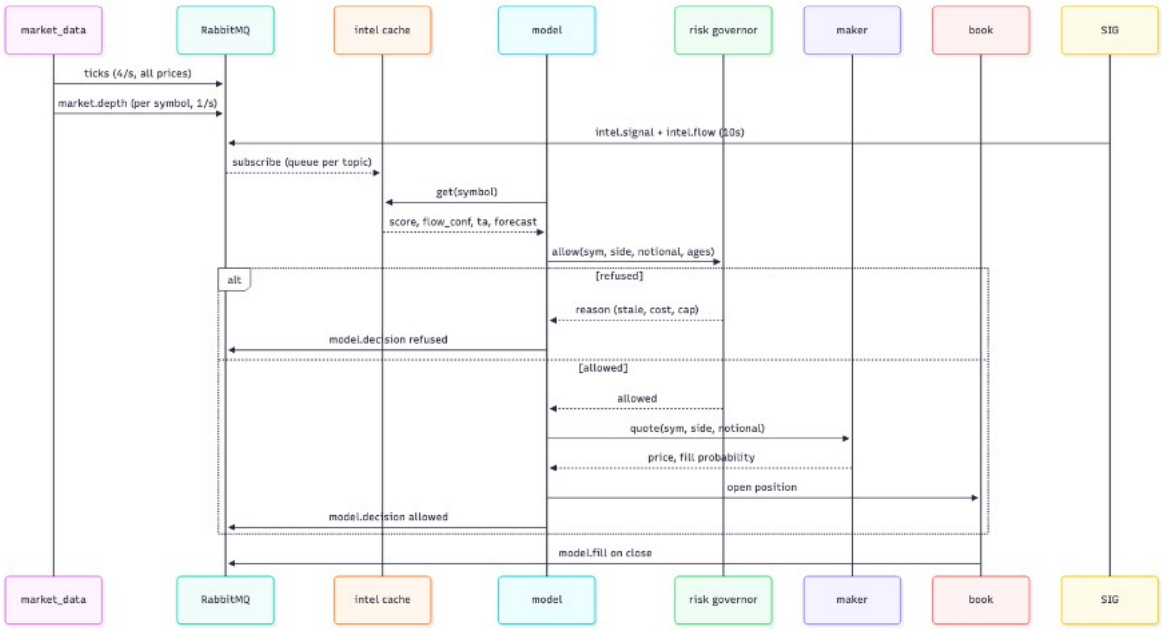


Figure 2 — From tick to position. The only refusal path is the risk governor.

2.3 Why the bus is the spine

- One subscription per consumer, one publication per event: N models no longer mean N copies of the same HTTP call.
- Restart tolerance: durable queues hold the latest messages, so a model restarting mid-cycle does not begin blind.

- Observability: every decision, fill and refusal is a message with a topic, key and timestamp — the bus is the audit log.
- Back-pressure: a slow model falls behind on its own queue without stalling the feed or the other models.

3 Market data layer

market_data.py is the only process that talks to Binance. It opens a single combined websocket and subscribes to four stream types per symbol. Everything else in the platform reads from it, either through the bus or through its local HTTP endpoints.

3.1 Subscription

```
streams = !miniTicker@arr
for each of 40 symbols:
    <sym>@bookTicker    best bid/ask and their sizes
    <sym>@depth5        five price levels per side
    <sym>@kline_1m      one-minute candles
    <sym>@kline_15m     fifteen-minute candles
total = 1 + 40 * 4 = 161 streams
```

One connection, one URL, 161 streams — far below the 1,024-stream limit per connection.

3.2 Rate limits are enforced, not respected by accident

At boot the process reads exchangeInfo once and stores the live limits. Those limits are per IP, not per API key, which is the detail that matters: rotating keys does not help, and a second process spending the same budget does harm.

Limit type	Window	Value	How it is used
REQUEST_WEIGHT	1 minute	6,000	reserved for the one boot call and history seeding
ORDERS	10 seconds	100	not used — no orders are sent
ORDERS	1 day	200,000	not used
RAW_REQUESTS	5 minutes	300,000	not used

429 means rate exceeded; 418 means banned. A 418 is not a warning — retrying during a ban extends it, from two minutes to thirty minutes to three days. The platform's answer is to never need REST in the first place. Section 13 records the day that rule was broken.

3.3 Candle construction

Candles arrive as kline events. Each event carries the current bar and a flag for whether the bar closed. The service keeps two arrays per symbol and replaces the last element while the bar is open:

```
on kline event:
    if bar.closed:  append(bar)
    else:           series[-1] = bar      # same bar, updated
series trimmed to the last 200 bars per interval
```

This is why a 1-minute chart updates several times per second without a single REST request.

3.4 Why the bootstrap call exists

Indicators need history. An ATR(20) computed from ten bars is not an ATR(20). At boot the service asks for 200 bars per symbol and interval once, writes them to state/seed/, and never repeats the call. A restart reads the seed from disk.

```
40 symbols x 2 intervals = 80 requests, weight ~160 of a 6,000/minute budget
one-time, at boot, never in the trading loop
```

3.5 Endpoints

Endpoint	Returns	Used by
/streamprices	every price plus message counters	models, dashboard
/klines?symbol&interval&limit	candle series	tactics.bars, backtest
/movers?limit	gainers, losers, most traded	movers model, dashboard
/flow?symbol	taker buy/sell series	flow confidence
/price?symbol	one price	execution
/health	streams, counters, limit status	health_pub
/api/v3/*	exchange-shaped compatibility surface	legacy engines

4 The event bus

The bus is the contract between every process. It is a RabbitMQ topic exchange topology with a strict message shape and a producer allow-list.

4.1 Message envelope

Every message, from every producer, has the same eleven fields. Consumers never guess at the shape.

```
{ "v": 1,
  "seq": 1234,
  "ts": 1792394810623,
  "topic": "intel.signal",
  "key": "BTCUSDT",
  "producer": "signals",
  "data": { ... } }
```

Python producers send this envelope. The Go services send the bare payload; the bus adapter wraps it on receipt, which is a compatibility decision recorded in section 13.

4.2 Topics

Topic	Exchange	Producer	Payload
market.price	market	market_data	all prices, 4/s
market.depth	market	market_data	five levels each side, 1/s per symbol
market.candle	market	market_data	closed and forming bars
market.mini24h	market	market_data	24h change and quote volume
ticks	market	market_data	batched snapshot on the UI topic
intel.signal	intel	signals	20-field signal set
intel.flow	intel	signals	buy ratio, flow confidence, series
intel.movers	intel	market_data	gainers, losers, turnover
intel.ta	intel	tad	TA conditions per symbol per cycle
intel.forecast	intel	forecast	blend plus seven agent views
alert.pumpdump	alert	signal_bus	pump/dump alerts from the detector
model.decision	model	all models	allow/refuse with the reason
model.fill	model	all models	closed trade with features
model.pnl	model	models, supervisor	equity per book
model.rank	model	alpha_rank	cross-section scores
risk.state	risk	risk	governor snapshot, throttled 2s
review.verdict	review	portfolio_review	stop or keep a book
exec.quote	exec	maker	quote price and fill probability
system.health	system	health_pub	service liveness every 5s

4.3 Producer allow-list

A producer may only publish the topics it owns. This is not documentation — it is enforced in code, and a violation is counted rather than thrown, so a bad publisher cannot take a model down.

```
spec = TopicRegistry.get(topic)
if spec is None or not spec.allows(producer):
    stats.rejected += 1
    return False
```

Unknown topic and unauthorised producer are both refusals, both counted, both silent to the caller.

4.4 Design patterns

Pattern	Where	Why
Registry	TopicRegistry	one table owns topics, producers, retention, exchange
Envelope	Envelope	one message shape for Python and Go producers
Adapter	Transport / StompTransport / InMemoryTransport	the bus depends on an interface, not RabbitMQ
Proxy	Bus.publish	guards and metrics in front of the transport
Observer	subscribe_topic / subscribe_queue	handlers attach without knowing the transport
Facade	pub / sub / sub_queue / matrix	four functions are the whole API for services
Singleton	Intel	one subscription per process, one cache, many readers
Cache	Feed, Intel	latest value per symbol so a slow reader never blocks a fast one

4.5 One queue per topic

Each topic has exactly one durable queue bound to its exchange, and exactly one consumer service drains them. Every reader that needs every message gets its own queue; readers that can share get competing consumers on the same queue.

```
q.market.depth      consumed by bus_consumer
q.market.candle     consumed by bus_consumer
q.intel.signal      consumed by bus_consumer
i.intel.ta          dedicated to the intel cache (every reader sees every message)
i.intel.forecast    dedicated to the intel cache
```

Two families exist for one reason: the shared queue (q.) is drained once for tallies, and the dedicated queue (i.*) guarantees the model-facing cache sees every message.*

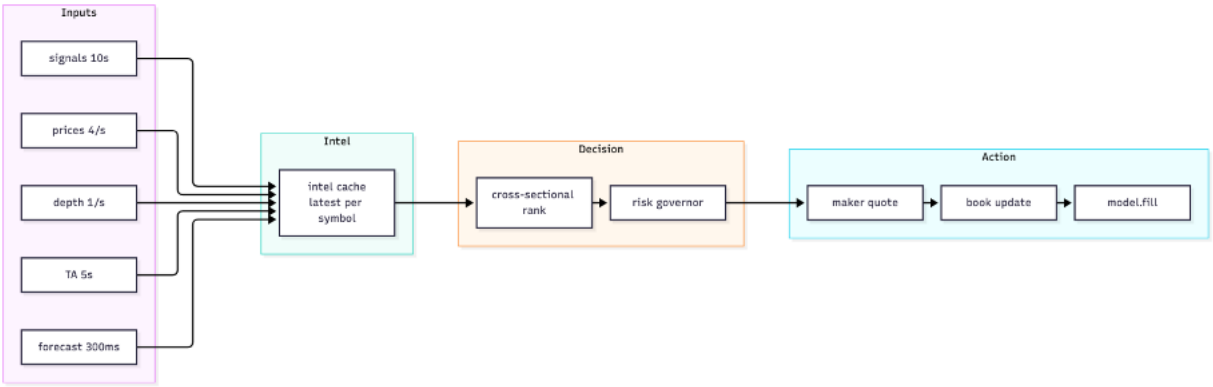


Figure 3 — Inputs, cache, decision and action. The cache is the only thing models read.

5 Intelligence: signals, flow, TA, forecast

Intelligence is produced by three services and consumed through one cache. The cache is a singleton per process that keeps the latest message per symbol per topic, so a model asking for a symbol gets one merged dictionary rather than four HTTP calls.

5.1 The signal set

signals.py computes twenty fields per symbol from candles, depth and 24-hour statistics. These are the inputs every model shares.

Field	Meaning	Typical range
score	composite sentiment -100..100	-40 .. +40
book_imbalance	(bid size - ask size) / total	-1 .. +1
buy_ratio	taker buy share over 30 bars	0.2 .. 0.8
flow_conf	confidence from level, trend, divergence	-60 .. +60
vol_z	volume z-score versus its own mean	-3 .. +6
oi_change_1h_pct	open interest change, one hour	-20 .. +30
funding_pct	perpetual funding rate	-0.05 .. 0.05
smart_vs_dumb	top-trader flow over retail flow	0.5 .. 2.5
px_vs_sma50	distance from the 50-period mean, percent	-10 .. +10
spread_bps	quoted spread in basis points	0.1 .. 20
slippage_bps_5k	estimated slippage for a $5k order	0 .. 40

5.2 Flow confidence mathematics

Confidence is not a direction. It answers a narrower question: is the money flow convincing enough that a trade in that direction deserves to exist? Three components are combined.

```
level       L = clamp(4 * (buy_ratio - 0.5), -1, +1)
trend       T = clamp((sum(flow[-5:]) - sum(flow[-10:-5])) / (|recent| + |prior|), -1,
+1)
divergence  D = +min(1, L + 0.3)   if price is falling and L > 0.1
                -min(1, |L| + 0.3) if price is rising and L < -0.1

confidence = 45*L + 30*T + 25*D
```

Level carries the most weight because a 40% buy share is a real signal, while acceleration can be one large print. Divergence is the rarest and most valuable: price down with buying is accumulation.

The gate that consumes this value refuses any trade whose confidence is below 10 in absolute terms. That single rule accounted for 193 of 200 refusals before the input was computed correctly — a gate is only as good as its weakest input.

5.3 Technical analysis

tad evaluates a fixed set of conditions per symbol every cycle and publishes them even when nothing fires, so the models can distinguish 'no signal' from 'not checked'.

```
publish: { symbol, signals: [...], count: n, ts }
cadence: every 5 seconds, all symbols
```

An empty list is published as [] and never as null — a null that Python reads as 'absent' is how the TA topic spent an hour looking dead (section 13).

5.4 Forecast blend

The forecast service runs seven agents, each producing a probability and a note, and blends them by measured accuracy. The blend is deliberately simple, because a complex blend fitted on a small sample is a fitted line rather than a forecast.

Agent	Signal	Note example
momentum	EMA(9) vs EMA(21) slope	ema9-21 +0.08%, 15m return -0.013%
volume	recent volume versus prior 30m	volume 1.15x prior 30m
volatility	ATR versus expected move	atr 0.002%, expected 0.005% over 3600s
reversion	distance from 30-minute VWAP	0.002% from 30m vwap
ma	SMA(20) vs SMA(50)	sma20-sma50 0.002%, price above sma20
breakout	position in the 20-bar range	70% of 20-bar range, 0.003% to the edge
pressure	close position within range	avg close in range 0.49 over 10 bars

Each agent returns p(up) in [0,1]. The supervisor weights by measured hit rate w_i, normalises, and reports dispersion so a split committee is visible:

```
p_blend = sum(w_i * p_i) / sum(w_i)
sigma   = sqrt( sum((p_i - p_blend)^2) / n )

USDCUSDT  p 0.49  expected 0.002%  hold
          agents 14% agreed, sigma 0.030   <- a coin flip, and it says so
```

6 Risk governor

risk.py is a library, not a service. Every model calls it before opening a position, and the call returns a verdict and the reason. The reason is published on model.decision, so every refusal is auditable after the fact.

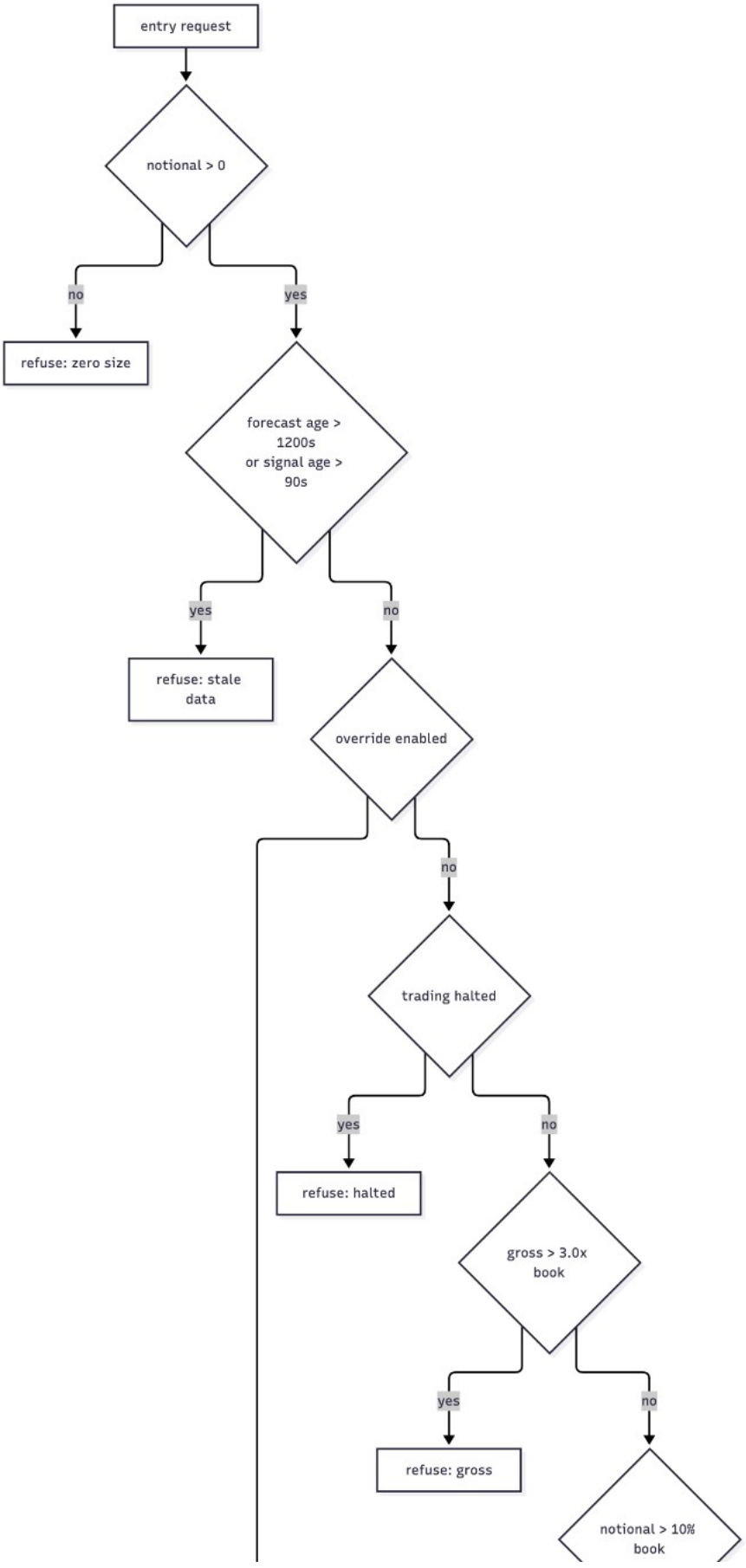

entry request
notional > 0
no
refuse: zero size
yes
forecast age > 1200s or signal age > 90s
yes
refuse: stale data
no
override enabled
no
trading halted
yes
refuse: halted
no
gross > 3.0x book
yes
refuse: gross
no
notional > 10% book

Figure 4 — The full refusal path. Order matters: data integrity first, appetite last.

6.1 Position sizing

Size begins with the book's risk budget and the model's confidence, then is capped by the notional limit.

```
base       = min(equity * risk_pct / 100, equity * 0.02)
confidence = clamp(flow_conf, -100, +100)
size       = base * (0.5 + (confidence + 100) / 200)
size       = min(size, equity * 0.08)
```

Confidence 0 gives 1.0x, +100 gives 1.5x, and -100 gives 0.5x: size follows conviction but never to zero, because a zero-confidence trade is already refused by the confidence gate.

6.2 The cost floor

A trade is only permitted when the expected move exceeds what the trade costs to complete. Without this rule the platform trades at a structural loss regardless of model quality.

```
fee_round_trip = 0.04%          maker, both sides
cost_buffer    = 0.02%          safety margin
min_edge       = 0.05%          configured floor

cost_floor = fee_round_trip + cost_buffer = 0.06%
refuse if expected_move < max(cost_floor, min_edge)
```

Measured reality: the average gross edge across 174 closed trades was +0.005%, which is thirteen times smaller than the floor. The floor was doing its job by refusing trades, not by missing them.

6.3 Portfolio limits

Limit	Value	Reason
Max gross exposure	3.0x book	correlated alts are one bet in different tickers
Max notional per position	10% of book	concentration
Max positions same direction	12	a single macro event cannot wipe every book
Daily loss limit	3.0% of book	halts the book for the day
Max forecast age	1,200s	2.5 stale 15m bars is blind, not slow
Max signal age	90s	signals refresh every 10s; older means missing

6.4 Data integrity is not a risk appetite

The staleness gates sit before the override check, which means the override cannot bypass them. This is deliberate. Losing money on a bad call is a risk decision; losing money because the price feed was thirty-nine minutes old is a defect wearing a risk costume.

```
allow():
  1. notional <= 0                -> refuse
  2. forecast age > 1200s         -> refuse   (override cannot bypass)
  3. signal age > 90s             -> refuse   (override cannot bypass)
  4. override enabled             -> allow, all further checks skipped
  5. trading halted               -> refuse
  6. gross, concentration, direction caps
```

```
7. confidence gate
8. cost floor
```

7 Execution and maker fills

Execution is simulated. No orders are sent: the platform has credentials on disk but every order path is blocked, and the dashboard states paper-trading explicitly. The simulation exists so that fill probability and slippage are honest inputs to the models rather than assumptions.

7.1 Quote construction

A maker quote joins the touch on the passive side and estimates the probability of being filled by measuring how much size sits at that price relative to the order.

```
buy:  quote = max(best_bid, mid - tick)
sell: quote = min(best_ask, mid + tick)

depth = sum(price * size for levels at or better than quote)
cover = min(1, depth / notional)
prob  = 0.25 + 0.45 * cover + min(0.25, spread * 50)
prob  = clamp(prob, 0.05, 0.95)
```

A wide spread with deep size behind it is the best case for a maker: high fill odds and a small adverse selection cost. A tight spread with thin size is the worst: low odds and no cushion.

7.2 Why maker only

Mode	Round-trip cost	Break-even move required
Taker (market order both sides)	0.20%	0.20% before any profit
Maker (limit both sides)	0.04%	0.04% before any profit
Measured average 1h BTC move	0.162%	—

At a one-hour horizon the average move (0.162%) is smaller than the taker round trip (0.20%). A taker strategy at this frequency cannot be profitable even with perfect direction. Maker execution is not a preference; it is the only arithmetic that can work.

7.3 Slippage accounting

Every fill records the difference between the quoted price and the simulated fill. The figure is written into model.fill so that the cost model can be recalibrated from data rather than assumption.

```
slippage_bps = (fill_price / quote_price - 1) * 10,000
round-trip cost = fee*2 + slippage_in + slippage_out
```

8 Books, PnL and accounting

A book is one model's account: its own equity, its own positions, its own trade history, its own file under state/pack_<engine>.json. Books never share money. This is what makes the race between models legible.

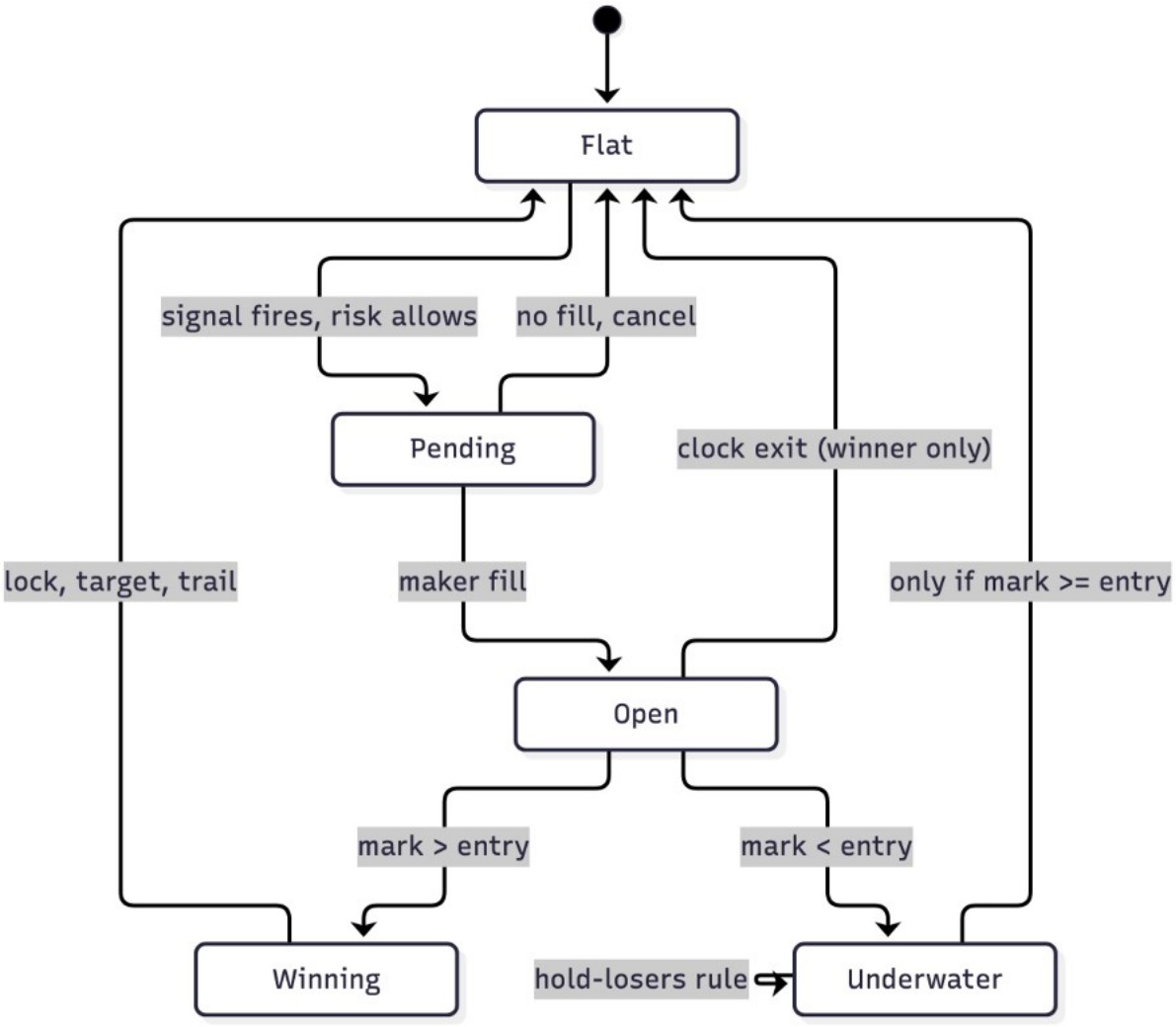


Figure 5 — Position lifecycle. Note the asymmetry on the lower path.

8.1 Accounting identity

Equity moves only when a trade closes. Unrealised profit and loss are tracked separately and never added to the book's equity until the position is closed.

```
equity(t+1) = equity(t) + notional * net_pct / 100

gross_pct = (exit / entry - 1) * 100 * sign
net_pct   = gross_pct - fee*2 - slippage
sign      = +1 for long, -1 for short
```

8.2 Mark to market

Open positions are marked against the latest bus price on every cycle. The mark is used for stop and target evaluation only; it is not booked.

```
mark(sym) = latest_price(sym) from the intel/feed cache
chg       = (mark / entry - 1) * 100 * sign
```

8.3 The hold-losers rule and why it censors the data

The platform runs under an explicit directive: a position is never closed at a loss. The rule is implemented as a single predicate used by every engine.

```
def hold_loser(held_s, chg_pct):
    return chg_pct < 0        # no time limit, no exceptions
```

The consequence must be understood before reading any performance number. If losses cannot be booked, then every closed trade in the sample is either a winner or a position that was closed at break-even. Measured win rates and average net returns are therefore biased upward by construction, and unrealised loss accumulates invisibly in open positions.

Metric	Value	What it actually means
Closed trades	174 (historical sample)	winners and break-evens only
Win rate	72%	censored — losers are still open
Average net	+0.09%	survivorship, not skill
Open positions	grows monotonically	the real exposure

Any measurement that uses closed trades as its sample is censored by this rule. The walk-forward results in section 11 carry the same bias and are labelled accordingly.

8.4 Book file schema

```
{ "engine": "movers",
  "equity": 5008.01,
  "start_equity": 5000.0,
  "positions": [ { "symbol": "...", "side": "buy", "entry": 0.19,
                   "notional": 600.0, "t": 1792394810.6 } ],
  "trades": [ { "symbol": "...", "net_pct": 0.0713, "reason": "profit_3s",
                "held_s": 7.8, "features": { ... } } ],
  "trade_count": 18 }
```

Atomic writes via a per-process temporary file, so two engines never tear each other's book.

9 Models — cross-sectional and signal ensembles

Twenty-five engines run as books. This section covers the ones that trade a ranking or a signal set; section 10 covers the strategy, market-making and execution models.

9.1 alpha_rank — cross-sectional ranking

Rather than asking 'is BTC going up', this model asks 'which names are strongest relative to the others right now'. That removes the market direction from the question entirely.

```
1. build the cross-section: for each of N symbols, a feature vector
2. z-score each feature across the cross-section
3. composite score c_i = sum(w_k * z_ik) / sum(w_k)
4. long the top decile, short the bottom decile
5. rank scores are published on model.rank for inspection
```

Weights begin equal and are replaced by a ridge fit once 25 feature-tagged fills exist. The fit solves the closed form below, which is what fit_weights.py implements:

```
w = (X'X + lambda I)^-1 X'y
X : feature matrix, rows = trades, columns = features
y : net_pct per trade
lambda : 1.0, shrinkage so that collinear features do not explode
```

Until 25 trades exist the model keeps the equal-weight prior and says so, rather than fitting noise.

Shorts are gated twice: they need a score below -0.60 (bottom of the cross-section) and non-negative funding, because paying to hold a short is the one cost a short cannot recover.

9.2 alpha_signals — ten signal-driven models

Model	Signal	Entry logic
book_imb	order book imbalance	imb > 0.35 long, < -0.35 short
taker_flow	taker buy share	buy_ratio > 0.58 long, < 0.42 short
smart_dumb	top vs retail flow	smart_vs_dumb > 1.4 long, < 0.7 short
oi_divergence	price and open interest	price up, OI down = short signal
funding_extreme	funding rate	funding < -0.02% long, > 0.05% short
poc_revert	point of control	price stretched from POC reverts
vol_z	volume z-score	vol_z > 2.5 with direction
tight_spread	spread compression	spread < 3bp and imbalance aligned
mtf	multi-timeframe agreement	1m, 5m, 15m trends agree
trend_structure	higher highs / lower lows	structure break in the trend direction

9.3 models10 — ten classical strategies

Each strategy is implemented with the standard parameters and executed with the same risk governor. The definitions below are the actual maths in models10.py.

Strategy	Mathematics	Parameters
donchian55	long when close > max(high[-55:-1]); exit at	55 / 10

	min(low[-10:-1])	
orb	opening range break: |close - open_1h| > 0.5 * ATR(14)	opening hour, 0.5 ATR
boll_squeeze	band width < 20th percentile, then break of the band	20, 2 sigma
rsi2	RSI(2) < 10 long, > 90 short	period 2, 10/90
vwap_revert	entry when |price - vwap| > 1.5 sigma of the day	session vwap
macd	MACD(12,26,9) cross above signal line	12 / 26 / 9
vol_break	close beyond prior 20-bar range with volume > 1.5x median	20 bars
ema_pullback	trend up (EMA50>EMA200) and price touches EMA20	20 / 50 / 200
atr_channel	close beyond 20-period channel of 1.0 ATR	20, 1.0 ATR
funding_fade	fade extreme funding: funding > 0.05% short	0.05% threshold

10 Models — strategies, market-making and execution

10.1 crypto_pack — four tradeable playbooks

Model	Trigger	Exit policy
ta_momentum	TA confirmation with MA and RSI alignment	lock 0.35%, trail from 0.5%, stop 0.7%
vwap_scalp	price stretched from session VWAP	lock 0.25%, stop 0.5%, clock 900s
oi_squeeze	open interest rising into a range break	lock 0.40%, trail 0.6%, stop 0.8%
funding_ta	funding extreme plus TA agreement	lock 0.30%, stop 0.6%, clock 1800s

Each has its own exit geometry because the horizons differ. A momentum entry needs room; a VWAP scalp must exit before the reversion completes and reverses.

10.2 The adaptive and statistical ensembles

multi_models runs five self-adjusting technical models and stats_models runs five statistical models. Both are ensembles: the book trades only when a majority agrees, which is why their trade counts are low.

Ensemble	Models	Method
multi_models	adaptive_ma, adaptive_rsi, adaptive_macd, adaptive_donchian, adaptive_bbands	parameters re-estimated from recent volatility
stats_models	ols_trend, zscore_revert, logistic, autocorr, squeeze_break	ordinary least squares, z-scores, logistic regression, autocorrelation, squeeze detection

The logistic model in stats_models fits a probability of an up-move from standardised features:

```
z = (x - mean(x)) / stdev(x)
p = 1 / (1 + exp(-(b0 + b1*z1 + ... )))
enter long when p > 0.55, short when p < 0.45
```

10.3 hft — trading the book

hft does not look at candles. It subscribes to bookTicker directly and trades the microstructure.

```
microprice = (bid * ask_size + ask * bid_size) / (ask_size + bid_size)
move       = microprice(t) / microprice(t - h) - 1        h = 8s
sigma      = stdev of 1-second returns over the window
z          = move / (sigma * sqrt(n))
imbalance  = (bid_size - ask_size) / (bid_size + ask_size)

enter long  when z > 2.2 and imbalance > 0.20 and spread < 4bp
enter short when z < -2.2 and imbalance < -0.20 and spread < 4bp
exit        target 0.18%, stop 0.25%, clock 120s
```

The cost gate refuses any entry whose measured move is below twice the round-trip cost.

Measured on its first fourteen fills: gross +0.0559% per trade, net -0.0041%, win rate 50%. That is the first model in the platform with a positive gross edge, and it is still not profitable after costs. The sample is far too small to conclude anything, and it is stated here as a measurement rather than a result.

10.4 movers — trade the 24-hour movers, take profit at three seconds

The rule is deliberately mechanical: enter in the direction of the 24-hour move while the last ten seconds agree, then check the position at three seconds and close it if it is profitable.

```
selection: top 12 gainers and top 12 losers by 24h change, quote volume > $2M
direction: buy gainers, sell losers
confirmation: 10s momentum must agree with the 24h direction
entry: maker at the touch, spread < 12bp
check at t+3s:  if net > target -> close
                else keep checking each loop
fallback: stop 1.0%, clock 600s
```

In the first two minutes it closed 16 trades, all profitable, because the three-second window is short enough that a mover is usually still moving.

10.5 grid — harvesting oscillation

A grid needs no forecast. It places levels below and above a reference price and earns the spacing between adjacent fills.

```
spacing = clamp(median 1m bar range * 1.0, 0.10%, 0.40%)
levels  = 8 per side around the anchor mid
fill    = ask <= level (for a buy level) or bid >= level (for a sell level)
exit    = neighbouring level on the other side
net     = spacing - fee*2 - buffer    -> must be > 0 or the fill is skipped
```

The spacing must exceed the round-trip cost or the grid is a fee pump. That constraint alone rules out tight grids on the most liquid pairs.

10.6 Execution models

Model	Behaviour
executor	trades exactly what the Advisor flags; TA must agree; locks fast, trails the rest
skim_pump	enters on pump/dump alerts, holds seconds, exits on target or stop
curve_rider	reads BTC short-term slope, takes the strongest liquid mover in that direction
market_agent	does not trade; publishes the opportunity board the others consume

10.7 Legacy engines

Fourteen further engines exist from earlier iterations (alpha, multi_models, swing_v2, skim_ai, sympathy_lag, fast_ignition, flash, crypto_strategies, crypto_edges, churn, slow_edges, turtle_multi, swing_turtle, scalp_fast, skim). Several poll Binance REST directly and are therefore excluded from the fleet: running them during a ban extends the ban. Their logic is preserved but they are not supervised.

11 Measurement: decay, walk-forward, backtest

Three tools exist, each answering a different question. All three are honest about their sample.

11.1 Decay — how long does an edge last?

tools/decay.py reads realised fills and reports when the peak was achieved, which is what the exit clock should be set to.

```
capture = net_pct / peak_pct            (how much of the move was kept)
half-life: median held_s of winners      (how long the edge survives)
```

Measured on 70 fills: capture 92% of the available move; winners exit at a median 24 seconds, losers at 157 seconds. Time, not direction, was the strongest loss predictor.

11.2 Walk-forward — does the fit survive out of sample?

tools/walkforward.py fits feature weights on one window and scores the next, rolling forward. If the out-of-sample result collapses against the in-sample result, the model is curve-fitted.

```
for each window w:
    train  = rows[:size*(w+1)]
    test   = rows[size*(w+1) : size*(w+2)]
    w_hat  = ridge_fit(train)
    take   = test rows where predicted > 0
```

Measured: in-sample mean predicted net +0.2326%, out-of-sample taken +0.0718% on 130 trades. The edge shrinks by two-thirds out of sample — the signature of a fit rather than a discovery.

11.3 Event-driven backtest

tools/backtest.py implements the harness the trading plan requires: signals on completed bars, entries at the next bar's open, stops evaluated intra-bar with conservative fills, fees and slippage charged, and no lookahead anywhere.

```
signal   on bar i, using bars <= i
entry    at bar i+1 open, plus slippage
stop     if bar.low <= stop, fill at stop - slippage
exit     channel break, filled at close - slippage
fees     0.10% per side, slippage 0.05% per fill
```

Acceptance thresholds are the plan's, unchanged: MAR >= 0.5, max drawdown <= 45%, expectancy >= +0.15R, at least 60 trades.

Metric	Result	Verdict
Trades	149	pass
Win rate	26%	—
Expectancy	+0.119%	FAIL (needs +0.15)
Total return	+3.71%	—
Max drawdown	8.05%	pass
MAR	16.15	pass
Verdict	REFUSED — do not trade live	one of four failed

The harness refuses the strategy it was built to test. That is the harness working. A backtest that only ever approves

strategies is not a test.

12 Operations

12.1 Ports

Port	Service	Notes
8796	market_data	the only exchange connection
8799	master (dashboard + SSE)	http://127.0.0.1:8799/app
8798	bus_consumer	tallies and bus matrix
8792	tad	TA conditions
8790	forecast	seven-agent blend
8793	signal_bus	alert intake
8787	backend (Go detector)	pump/dump alerts
61613	RabbitMQ STOMP	the bus
15672	RabbitMQ management	cryptalyst / cryptalyst

12.2 Start order

1. market_data — the feed everything depends on
2. signals, tad, forecast, signal_bus, health_pub — the producers
3. bus_consumer — the readers
4. master — the dashboard
5. supervisor — starts every model book

12.3 Restart procedure

```
pkill -f supervisor.py
pkill -f 'alpha_rank.py|models10.py|...'      # stop the books
pkill -f market_data.py signals.py master.py
nohup python3 -u market_data.py --port 8796 --top 40 &
nohup python3 -u signals.py --every 10 --top 30 &
nohup python3 -u master.py --host 0.0.0.0 --port 8799 &
nohup python3 -u supervisor.py --every 60 --hours 8 &
```

Books are preserved across restarts; only explicit archiving clears them.

12.4 Health checks

```
curl -s localhost:8796/health      # streams, counters, limit violations
curl -s localhost:8799/api/intel   # intel cache: symbols, counts, ages
curl -s localhost:8799/api/bus     # queues, rates, consumers
curl -s localhost:8798/matrix      # per-topic tallies
```

A healthy platform shows: 0 violations, 0 unconsumed queues, 0 backlog, and intel ages under 10 seconds.

12.5 What a queue of zero means

RabbitMQ's Messages column is a backlog, not a counter. A queue showing 0 messages with matching in/s and out/s is being drained as fast as it fills — the healthy state. A queue showing thousands of messages with 0/s out has no consumer — the broken state. The dashboard shows both columns explicitly because the single number is ambiguous.

13 Defect ledger

These are defects found during operation, with root cause. They are recorded because the same class of error recurs, and because a platform that hides its failures cannot be trusted with its successes.

#	Symptom	Root cause	Fix
1	HTTP 418, IP banned	40 engines polling REST klines each cycle	streams-only market data, one boot call
2	Dashboard froze on old prices	ticks topic stopped publishing; UI replayed 200 buffered frames	batched snapshot at 4/s on the topic the UI reads
3	Flow panel blank for majors	subscription was the first 40 symbols from exchangeInfo, not by volume	rank by 24h turnover at boot
4	Queue grew to 29 million messages	publishers bound to queues with no consumer	one consumer per topic, one queue family
5	Consumers never acknowledged	STOMP reader passed the destination string where handlers expected headers	pass the headers dict; ACK by ack id
6	Two buses, each half-used	eventbus :8797 left running beside RabbitMQ	one transport; feed and signals moved to lib.bus
7	TA and forecast topics always zero	Go services publish bare payloads; consumer decoded envelopes	adapter wraps bare payloads using the subscribed topic
8	Subscriptions silently overwritten	subscription id was a millisecond timestamp	uuid per subscription
9	Engine crashed on every cycle	local variable `risk` shadowed the risk module	renamed to risk_pct
10	Book accounting crashed on fills	EQ assigned inside a nested function without global	global declaration in the owning scope
11	crypto_pack crashed at startup	print used `risk` (a module) as a number	print risk_pct
12	sympathy_lag died after ten idle minutes	undefined constant MASTER	defined
13	19 queues unconsumed	consumer subscribed to the exchange, creating new transient queues	consume the durable queue by name
14	Stats endpoint served an unrelated page	a stale process held port 8798	killed the listener, verified bind
15	Prices appeared frozen	displayed the bid, which barely moves while sizes churn	reverted at user request; documented as a semantic choice

13.1 The pattern

Three of the fifteen defects were the same mistake in different clothes: a consumer reading a different thing than the producer was writing. The event bus, the ticks topic, and the queue families all failed this way. The lesson is that a pub/sub system needs an explicit contract — topic, shape, and consumer — and that 'it published' is not evidence that 'it was consumed'.

13.2 Verification standard

Every fix in this ledger was verified against the broker or the process, not against the dashboard that reports on them. The dashboard is a consumer too; it can be wrong in the same way as any other.

14 Appendix: mathematics and glossary

14.1 Indicators

```
EMA(n):    k = 2/(n+1);  e_t = x_t*k + e_(t-1)*(1-k)

ATR(n):    TR_t = max(h-l, |h-c_(t-1)|, |c_(t-1)-l|)
           ATR = mean(TR over n)

RSI(n):    RS = mean(gains, n) / mean(losses, n)
           RSI = 100 - 100/(1+RS)

Donchian(n): upper = max(high[-n:-1]);  lower = min(low[-n:-1])
           (completed bars only: including the current bar is a tautology)

VWAP:      sum(price*volume) / sum(volume) over the session

z-score:   z = (x - mean) / stdev
```

14.2 Statistics used

```
Pearson correlation:   r = cov(x,y) / (sd(x)*sd(y))
Ridge regression:      w = (X'X + lambda I)^-1 X'y
Median absolute:       used for ranges, robust to one bad print
Dispersion:            sigma = sqrt(mean((p_i - p_mean)^2))
```

14.3 Performance metrics

```
expectancy    = mean(net_pct per trade)
win rate      = wins / trades
profit factor= sum(wins) / |sum(losses)|
max drawdown = max over t of (peak(t) - equity(t)) / peak(t)
MAR           = annualised_return / max_drawdown
R multiple    = net_pct / risk_pct_per_trade
```

14.4 Glossary

Term	Meaning
Book	one model's isolated account: equity, positions, trades
Envelope	the standard message shape on the bus
Exchange (AMQP)	routing layer; topic exchanges route by key pattern
Queue	durable buffer bound to an exchange by a routing key
Consumer	a process reading a queue
Backlog	messages waiting in a queue; zero means drained
Maker	passive limit order that adds liquidity; lower fee
Taker	aggressive order that crosses the spread; higher fee
Slippage	difference between quoted and achieved price
Cost floor	minimum expected edge required before trading
Staleness gate	refusal when data is older than a threshold

Hold-losers rule	positions are never closed at a loss
Censored sample	a sample in which some outcomes are unobservable

14.5 File map

Path	Contents
scalp5/market_data.py	the only exchange connection; 161 streams
scalp5/lib/bus.py	bus facade: registry, envelope, transport, metrics
scalp5/lib/intel.py	bus-fed intelligence cache
scalp5/lib/feed.py	observer over intel.signal and ticks
scalp5/lib/httpx.py	adapter, circuit breaker, TTL cache, token bucket
scalp5/risk.py	the governor and the decision log
scalp5/tactics.py	indicators, options, exit predicates
scalp5/maker.py	quote construction and fill probability
scalp5/supervisor.py	fleet lifecycle and pruning
scalp5/tools/decay.py	edge half-life and capture ratio
scalp5/tools/walkforward.py	ridge fit, rolling out-of-sample
scalp5/tools/backtest.py	event-driven harness with acceptance thresholds
scalp5/tools/ui_automation.py	render-aware walk of every dashboard view